GRAPHQL MIT RUBY ON RAILS

Schritt 35 — Fragments

Unterrichtsskript 1:1 • Project Management API • Client Request

1. Gehören Fragments nur zu GraphQL oder zum Client-Request?

Ein Fragment ist GraphQL-Syntax und wird normalerweise auf der Client-Seite im GraphQL-Dokument bzw. Request geschrieben. Rails besitzt keinen eigenen Fragment-Resolver.

```
Fragment
→ gehört zur GraphQL-Sprache
→ Client schreibt es in Query/Mutation
→ Server erhält ein Dokument mit Fragment
→ GraphQL Engine validiert/expandiert es
→ Resolver verarbeitet die benötigten Felder
```

2. Ohne Fragment

```
query GetTasks($projectId: ID!) {
  tasks(projectId: $projectId, first: 10) {
    nodes {
      id
      title
      status
      priority
      dueDate
    }
  }
}
```

Wenn eine Mutation dieselben Felder benötigt, wird dieses Selection Set wiederholt.

3. Mit Fragment

```
fragment TaskFields on Task {
  id
  title
  status
  priority
  dueDate
}

query GetTasks($projectId: ID!) {
  tasks(projectId: $projectId, first: 10) {
    nodes {
      ...TaskFields
    }
  }
}
```

...TaskFields ist ein Fragment Spread und wiederverwendet das Selection Set.

4. Wo liegt das Fragment?

```
src/
└── graphql/
    ├── fragments/
    │   └── taskFields.js
    ├── queries/
    │   └── getTasks.js
    └── mutations/
        └── createTask.js
```

```
export const TASK_FIELDS = `
  fragment TaskFields on Task {
    id
    title
    status
    priority
    dueDate
  }
`;
```

Bei Axios/plain strings muss das Fragment in das GraphQL-Dokument eingebunden werden, das an den Server gesendet wird.

5. Echter Axios Request

```
const response = await axios.post(
  "/graphql",
  {
    query: `
      fragment TaskFields on Task {
        id
        title
        status
        priority
        dueDate
      }

      query GetTasks($projectId: ID!) {
        tasks(projectId: $projectId, first: 10) {
          nodes {
            ...TaskFields
          }
        }
      }
    `,
    variables: {
      projectId: "10"
    },
    operationName: "GetTasks"
  },
  {
    withCredentials: true
  }
);
```

Der HTTP-Body enthält query document, variables und operationName. Das Fragment befindet sich im query document.

6. Braucht Rails eine TaskFieldsFragment-Klasse?

Nein. Es gibt keine TaskFieldsFragment-Klasse und keinen task_fields-Resolver.

```
class TaskType < Types::BaseObject
  field :id, ID, null: false
  field :title, String, null: false
  field :status, Types::Enums::TaskStatusEnum, null: false
  field :priority, Types::Enums::TaskPriorityEnum, null: false
  field :due_date, GraphQL::Types::ISO8601Date, null: true
end
```

Rails muss lediglich den GraphQL-Typ mit den Feldern bereitstellen, die das Fragment verwendet.

7. Was bedeutet `on Task`?

```
fragment TaskFields on Task {
  id
  title
}
```

on Task bedeutet: Dieses Fragment gilt für den GraphQL-Typ Task. TaskFields kann nicht auf Project gespreadet werden.

8. Fragments können andere Fragments enthalten

```
fragment UserFields on User {
  id
  email
}

fragment TaskFields on Task {
```

```
  id
  title
  assignee {
    ...UserFields
  }
}
```

9. Fragments in Query und Mutation

```
query GetTask($id: ID!) {
  task(id: $id) {
    ...TaskFields
  }
}

mutation UpdateTask($input: UpdateTaskInput!) {
  updateTask(input: $input) {
    task {
      ...TaskFields
    }
    errors
  }
}
```

10. Reduziert ein Fragment SQL?

Nein, nicht automatisch. Ein Fragment organisiert nur das GraphQL Selection Set. Es führt weder automatisch Preloading noch N+1-Vermeidung oder Datenbank-Caching durch.

Wenn ein Fragment assignee auswählt, benötigt Rails weiterhin Dataloader/Preloading, falls sonst N+1 entsteht.

11. Fragment vs. Input / Variable / Operation Name

```
Input       → Daten, die der Client HINEINSENDET
Fragment    → Felder, die der Client ZURÜCKERHALTEN möchte
Variable    → dynamischer Wert, z. B. $projectId
Operation   → Name der Query/Mutation, z. B. GetTasks
```

12. Namenskonventionen

```
UserFields
ProjectFields
TaskFields

TaskSummaryFields
TaskDetailFields

ProjectSummaryFields
ProjectDetailFields
```

Fragments sollten zum UI-Bedarf passen. Ein riesiges Fragment, das überall verwendet wird, führt leicht zu unnötigen Feldern.

13. Professionelle Frontend-Struktur

```
src/
├── graphql/
│   ├── fragments/
│   │   ├── taskFragments.js
│   │   └── userFragments.js
│   ├── queries/
│   │   └── getTasks.js
│   └── mutations/
│       ├── createTask.js
│       └── updateTask.js
└── api/
    └── graphqlClient.js
```

```
export const TASK_SUMMARY_FIELDS = `
```

```
  fragment TaskSummaryFields on Task {
    id
    title
    status
    priority
    dueDate
  }
`;

export async function graphqlRequest({
  query,
  variables,
  operationName
}) {
  const response = await axios.post(
    "/graphql",
    { query, variables, operationName },
    { withCredentials: true }
  );

  return response.data;
}
```

14. Merksätze zu Schritt 35

```
Fragment
→ gehört zur GraphQL-Sprache

Wo wird es geschrieben?
→ normalerweise im Frontend/Client

Wo wird es gesendet?
→ im GraphQL-Dokument des Requests

Was muss Rails tun?
→ keine Fragment-Klasse erstellen
→ Schema/Type muss die verwendeten Felder besitzen

Wozu dient es?
→ Selection Sets wiederverwenden

Reduziert es SQL?
→ nein

Für Query und Mutation?
→ ja
```

```
CLIENT
│
├── Operation: GetTasks
├── Variables: projectId
└── Fragment: TaskFields
        │
        ▼
GraphQL document
        │
        ▼
POST /graphql
        │
        ▼
Rails GraphQL Schema
        │
        ▼
Fragment gegen TaskType validieren
        │
        ▼
Resolver
        │
        ▼
Response
```